MODULE 27 · DOMAIN 4 - TROUBLESHOOTING AND OPTIMIZATION

Troubleshooting – Errors, Throttling and Retries

Which errors to retry, how to back off, and why the same fix makes some outages worse.

DOMAIN 4 of 4 EXAM WEIGHT 18% MODULE 27 of 34

admin@sthithapragna.com

Retrying a **transient** failure is correct. Retrying a **permanent** one wastes time, and retrying a **throttle** badly makes the outage worse.

The error taxonomy

- **4xx means the client was wrong.** Bad parameters, missing permission, resource not found.
- **5xx means the service failed.** The same request might succeed on the next attempt.
- **429 is the exception**, a 4xx that should be retried, because the request was valid.
- **408 request timeout** is the other 4xx worth retrying.
- So the rule is: retry 5xx, retry 429 and 408, do not retry other 4xx.

> ON THE EXAM
>
> Retrying a 403 is the most common wasted retry. The permission will not appear between attempts, and the retries delay the error reaching whoever can fix it.

Retryable and not

CATEGORY	EXAMPLES
Retry: throttling	ThrottlingException, TooManyRequests, SlowDown
Retry: transient service	500, 502, 503, 504, InternalServerError
Retry: transient network	Connection reset, socket timeout, DNS failure
Do not retry: client	ValidationException, InvalidParameterValue
Do not retry: auth	AccessDenied, expired credentials
Careful: conditional	ConditionalCheckFailedException

ON THE EXAM

The last row matters: a conditional check failure is not an error to retry blindly. Something else changed the item.

Exponential backoff

- Wait longer after each failed attempt: roughly 50 ms, 100 ms, 200 ms, doubling each time.
- **Without jitter, every client retries at the same moment.** The overload repeats in waves.
- **Jitter** randomises each delay, spreading retries across the window.
- Full jitter picks a random value between zero and the computed backoff.
- A **maximum backoff** caps the delay so it does not grow indefinitely.

> ON THE EXAM
>
> Backoff alone is not enough. A thousand clients all doubling from the same start still retry in synchronised waves; jitter is what breaks the synchronisation.

The SDK already does this

- Every AWS SDK retries retryable errors automatically, with exponential backoff and jitter.
- So a visible failure has usually **already been retried** two or three times.
- Wrapping SDK calls in your own retry loop **multiplies** the attempts.
- Three SDK attempts inside three of your own is nine, with unpredictable total latency.
- Configure the SDK instead: retry mode and max attempts are both settings.

> ON THE EXAM
>
> The multiplication is the practical danger. A Lambda function timing out at 30 seconds can spend all of it inside nested retry loops that were each individually reasonable.

The three retry modes

MODE	BEHAVIOUR
`standard`	Exponential backoff with jitter, plus a retry quota. The recommended default
`adaptive`	Everything in standard, plus client-side rate limiting. Specialised, and experimental
`legacy`	Older, SDK-specific behaviour with no retry quota. For backward compatibility only

ON THE EXAM

Adaptive is not simply 'better standard'. It rate-limits the client itself, so in a multi-tenant application one tenant's throttling can delay another's requests.

Max attempts

- **Max attempts defaults to 3**: one initial request plus two retries.
- Setting it to **1 disables retries entirely**, which is occasionally what you want.
- **DynamoDB clients default to 4**, with a shorter base delay, matching its low-latency profile.
- Configurable in code, by **AWS_MAX_ATTEMPTS**, or in the shared config file.
- Code beats environment variable, which beats the config file.

> ON THE EXAM
>
> Max attempts counts total attempts, not retries. A value of 3 means three requests reach the service at most, which is a common off-by-one misreading.

The retry quota

- Standard mode holds a **token bucket** for retries across the client.
- Each retry deducts tokens. Successful requests replenish them.
- When tokens run out, the SDK **returns the error without retrying**.
- So during a sustained outage the application fails fast rather than waiting through retries.
- **Legacy mode has no retry quota**, which is the main reason it is not recommended.

> ON THE EXAM
>
> The quota converts a slow, retrying failure into a fast one. During a real outage, failing in 100 ms is far better than every request taking six seconds to give up.

Currency note: retry behaviour is changing

- **The examinable model:** exponential backoff with jitter, three attempts, standard mode.
- AWS announced updated retry behaviour, opt-in now via **AWS_NEW_RETRIES_2026**.
- It changes backoff timing, retry quota costs and the DynamoDB defaults.
- It also switches **Java, Python, Ruby, PHP, C++ and the CLI** from legacy to standard.
- **Default rollout is November 2026.** Explicitly set values are preserved.

> ON THE EXAM
>
> The principle does not change: exponential backoff with jitter, and do not retry client errors. Only the specific defaults are moving.

How each service tells you it is throttling

SERVICE	WHAT YOU GET
AWS Lambda	429 TooManyRequestsException, with a Throttles metric
Amazon DynamoDB	ProvisionedThroughputExceededException, or ThrottlingException
Amazon API Gateway	429 Too Many Requests
Amazon S3	503 Slow Down
AWS KMS	ThrottlingException. Quotas are per account and Region
Amazon SQS and Kinesis	ProvisionedThroughputExceededException on Kinesis shards
AWS STS	ThrottlingException on high AssumeRole volume

ON THE EXAM

S3 is the odd one: it signals throttling with a 503 rather than a 429, so it looks like a server error and is retried as one. The effect is the same.

What to do about throttling

- **Retry with backoff and jitter first.** Brief throttling is normal and self-correcting.
- **Reduce the request rate:** batch, cache, or buffer through a queue.
- **Spread the load:** better partition keys, more prefixes, more shards.
- **Raise the limit:** provisioned capacity, reserved concurrency, or a quota increase.
- Retrying harder without any of the above turns a throttle into a sustained outage.

> ON THE EXAM
>
> The order is deliberate. Retrying is the immediate response, and if throttling is sustained the answer is fewer requests or more capacity, never more retries.

Idempotency

- **Every retry mechanism can process the same request twice.** That is not a bug.
- A timeout is the worst case: you do not know whether the operation succeeded.
- An **idempotency key** lets the receiver recognise and ignore a repeat.
- In DynamoDB, a **conditional write** gives idempotency directly.
- Store the key and the result, so a repeat returns the original outcome.

> ON THE EXAM
>
> The timeout case is why this matters. The request may have succeeded and the response been lost, so a retry without an idempotency key charges the customer twice.

Connection pool exhaustion

- A pool of connections is finite. Under load, requests queue waiting for one.
- That surfaces as a **timeout**, not as a connection error, which misdirects the investigation.
- For Lambda and a relational database, this is module 10's problem: **RDS Proxy** or the Data API.
- A connection **not returned** to the pool leaks one each time, so it degrades gradually.
- The signature is errors that scale with concurrency and vanish when traffic drops.

> ON THE EXAM
>
> 'Fine in testing, fails in production' is usually this. A single-threaded test never exhausts a pool, so the fault only exists at concurrency.

Timeouts are layered

- A request passes through several timeouts, each set independently.
- Client, then API Gateway, then Lambda, then the SDK, then the downstream service.
- **An inner timeout longer than an outer one is useless.** The outer fires first.
- An SDK timeout of 60 seconds inside a 30-second function never takes effect.
- Each layer should time out **before** the layer outside it, leaving room to handle the failure.

> READ THIS TWICE
>
> This is the commonest timeout misconfiguration, and it is invisible until something is slow. The inner timeout was meant to produce a clean error and never gets the chance.

The timeout numbers

WHERE	LIMIT
AWS Lambda	15 minutes maximum, 3 seconds default
API Gateway integration	29 seconds. See module 11 for what has changed
Application Load Balancer idle	60 seconds by default, configurable
Step Functions Standard	1 year. Express, 5 minutes
SQS visibility timeout	12 hours maximum, 30 seconds default
SDK socket timeout	SDK-specific, and configurable per client

ON THE EXAM

The 3-second Lambda default and the 30-second SQS default are both shorter than people expect, and both cause failures that look like something else.

Timeout, cold start or throttle?

- A **timeout** shows Task timed out after N seconds in the logs, and Duration equals the limit.
- A **cold start** shows an Init Duration in the REPORT line, and only on some invocations.
- A **throttle** produces no function logs at all, and increments the Throttles metric.
- Throttling is the confusing one, because the function never ran and wrote nothing.
- So absence of logs is itself a diagnostic signal, not a logging problem.

> ON THE EXAM
>
> That last line is worth holding onto. When invocations are clearly failing and the log group is empty, look at Throttles before you look at the code.

Poison pills

- A message that always fails is retried until it expires, or forever on an ordered source.
- On a **Kinesis or DynamoDB Stream shard**, it blocks every message behind it.
- **maxRecordAge** and **maxRetryAttempts** on the event source mapping bound it.
- **bisectBatchOnFunctionError** splits a failing batch to isolate the bad record.
- For SQS, **maxReceiveCount** and a DLQ do the same job.

> ON THE EXAM
>
> The ordered-source case is the severe one: an SQS poison pill delays one message, while a Kinesis poison pill stops the entire shard until it is bounded.

Failing fast

- A **circuit breaker** stops calling a dependency that is consistently failing.
- After a threshold of failures it opens, rejecting calls immediately for a period.
- It then half-opens, allowing one probe through to test recovery.
- The SDK's **retry quota** is a simpler form of the same idea.
- A **fallback** returns degraded but useful data instead of an error.

> ON THE EXAM
>
> The argument is that during a dependency outage, retrying consumes your own capacity too. Failing fast preserves the parts of your system that still work.

Service quotas

- **Service Quotas** lists every limit, whether it is adjustable, and your current usage.
- Some quotas are **hard** and cannot be raised. API Gateway's burst is one.
- Adjustable quotas are raised through a request, which is not instant.
- A **CloudWatch alarm on quota utilisation** warns before you hit the ceiling.
- Quotas are usually **per account and Region**, so one workload can starve another.

> ON THE EXAM
>
> Alarming on quota utilisation is the underused part. Discovering a hard limit during an incident is far worse than discovering it at 80% on a Tuesday.

Where to look

- **Metrics first.** Errors, Throttles and Duration tell you the category in seconds.
- **Then the trace.** X-Ray shows which component and which call.
- **Then the logs** for that specific request, found by trace or correlation ID.
- Starting with logs is the slow path, because you do not yet know where to look.
- **CloudTrail** if the question is what changed, rather than what is failing.

> ON THE EXAM
>
> Metrics, trace, logs, in that order. Each step narrows the search, and jumping straight to logs means searching everything without knowing what for.

Symptom to cause

SYMPTOM	USUAL CAUSE
Failing invocations, empty log group	Throttling, or missing logs permission
Duration exactly equals the timeout	A timeout, not an application error
Intermittent slowness, only some requests	Cold starts, or a hot partition
Every request fails after a deployment	Configuration or permission, not capacity
Errors rise only under load	Throttling, or a connection pool exhausted
Duplicate records downstream	Retries without idempotency
One consumer stuck, others fine	A poison pill on an ordered shard

ON THE EXAM

Worth screenshotting. Most troubleshooting questions on this exam describe one of these rows, and the wrong answers are usually a different row's cause.

The failures that are not capacity

- **Permission failures** from module 15: AccessDenied, and no amount of retrying helps.
- Broad permissions still failing means an explicit deny, usually an SCP or a boundary.
- **Networking failures** in a VPC: no route, no NAT gateway, or a security group blocking.
- These typically present as a **connection timeout**, not a permission error.
- A VPC endpoint is needed to reach an AWS service privately without internet egress.

> ON THE EXAM
>
> The confusable pair: a permission failure returns AccessDenied quickly, while a networking failure hangs until the socket times out. The timing tells you which.

Retry storms

- A brief failure causes retries, which add load, which causes more failures.
- **Nested retries multiply**: three layers of three attempts is twenty seven requests.
- Jitter, retry quotas and circuit breakers all exist to break this loop.
- Retry at **one layer**, ideally the outermost one that can act on the failure.
- A queue in front of a struggling service converts a retry storm into a backlog.

> ON THE EXAM
>
> That last point is the architectural answer. A backlog is a problem you can watch drain; a retry storm is a problem that grows while you watch it.

Six scenarios. For each, decide first: is this capacity, configuration, or correctness?

> A DynamoDB table in on-demand mode serves an event ticketing application. During a popular release, writes fail with ProvisionedThroughputExceededException even though the table has no provisioned capacity configured.
>
> All writes during the spike use the event ID as the partition key, and one event accounts for almost all of them.

What is the cause?

A On-demand mode has a hard account-wide write limit that has been reached

B The SDK is retrying too aggressively and causing its own throttling

C A hot partition. All writes target one partition key, exceeding the per-partition limit

D The table needs a global secondary index to spread the writes

◆ PAUSE AND DECIDE

What is the cause?

A Account-wide limit

On-demand scales to very high throughput and this is nowhere near an account ceiling. It also would not correlate with one event ID.

B Aggressive SDK retries

Retries amplify throttling but cannot cause it from nothing. Something has to be exceeding a limit first.

C A hot partition

Per-partition limits apply regardless of capacity mode. All writes hitting one partition key exceed 1,000 WCUs for that partition while the table is barely loaded.

D Add a GSI

A GSI serves different query patterns. It does not redistribute writes to the base table, and it would add write cost.

THE SIGNAL

The clue is that on-demand mode does not remove per-partition limits. Module 7 made this point: capacity mode changes the table's ceiling, never the partition's.

> A Lambda function consumes an SQS queue and charges a customer's card for each message. Finance reports that a small number of customers were charged twice on the same day. The function has no errors in CloudWatch and the DLQ is empty.
>
> The payment provider's API occasionally takes longer than usual to respond.

What is the most likely cause and fix?

A. The charge is not idempotent. A timed-out call succeeded and was retried, so add an idempotency key

B. The DLQ is misconfigured and is silently replaying messages

C. SQS is delivering duplicates because the queue is not FIFO

D. The function timeout is too high and should be reduced

◆ PAUSE AND DECIDE

What is the most likely cause and fix?

A Not idempotent, add a key

A slow payment call times out on the client side after the charge succeeded. The retry charges again. An idempotency key makes the second call return the first result.

B DLQ replaying

A DLQ receives messages, it does not replay them. The scenario also states it is empty.

C Standard queue duplicates

The strongest distractor, and it is only half the story. At-least-once delivery is real, and the fix is still idempotency rather than FIFO, which would cripple throughput.

D Reduce the timeout

Would cause more timeouts and therefore more double charges, since the underlying operation still succeeds before the client gives up.

THE SIGNAL

The clue is the slow third-party API. A timeout is the one failure where you genuinely do not know whether the operation happened, which is exactly what idempotency exists for.

> An API Gateway REST API integrates with a Lambda function that calls a slow reporting database. Users report that some requests return 504 Gateway Timeout. The Lambda function's timeout is set to 60 seconds and its Duration metric shows successful invocations completing in up to 45 seconds.
>
> The function itself reports no errors.

What is happening?

A The Lambda function is being throttled and never runs

B The Lambda function is returning a malformed response

C The database connection is being refused

D API Gateway's 29-second integration timeout fires while the function is still running

◆ PAUSE AND DECIDE

What is happening?

A Throttling

Throttling returns 429, not 504. The Duration metric also shows the function running, so it clearly is being invoked.

B Malformed response

Module 11's fact: a bad proxy response produces 502, not 504. The two codes mean different things.

C Connection refused

Would produce a function error and appear in the logs. The scenario says the function reports no errors.

D The 29-second gateway timeout

Invocations run up to 45 seconds, well past the 29-second integration timeout. API Gateway gives up and returns 504 while the function carries on.

THE SIGNAL

The layered timeout problem from earlier. The function timeout of 60 seconds is longer than the gateway's 29, so the inner limit never takes effect.

> A fleet of 500 EC2 instances calls a downstream service. A brief 30-second outage of that service turned into a 20-minute one. Metrics show request volume to the service increased tenfold during the incident and stayed high after the underlying fault was fixed.
>
> Each instance retries failed calls five times with a fixed one-second delay.

What should be changed?

A. Increase the retry count so requests eventually succeed

B. Use exponential backoff with jitter, and add a retry quota or circuit breaker

C. Remove retries entirely so failures surface immediately

D. Add more instances to the fleet to spread the load

◆ PAUSE AND DECIDE

What should be changed?

A More retries

Directly worsens it. The tenfold volume increase is caused by retries, so adding more amplifies the storm that extended the outage.

B Backoff with jitter, plus a quota

Fixed delays synchronise 500 clients into waves. Jitter spreads them, backoff reduces the rate, and a quota or breaker stops retrying into a sustained failure.

C No retries

Overcorrects. Transient failures that would have self-corrected now become user-visible errors.

D More instances

Increases the number of clients generating retries. It makes the storm larger.

THE SIGNAL

'Fixed one-second delay' across 500 instances is the whole problem: every client retries at the same instant, five times, recreating the overload in waves.

> A Lambda function processes records from a Kinesis Data Stream. One shard has stopped making progress entirely, while the other shards continue normally. The iterator age for the stuck shard has been climbing for hours.
>
> The function throws an exception on one particular malformed record.

What should the team configure?

A. Increase the function's reserved concurrency so it can keep up

B. Add more shards to the stream to spread the load

C. Set maxRetryAttempts and maxRecordAge on the event source mapping, and enable bisectBatchOnFunctionError

D. Switch the stream to enhanced fan-out

◆ PAUSE AND DECIDE

What should the team configure?

A More concurrency

The shard is blocked, not slow. Kinesis processes one batch at a time per shard, so extra concurrency cannot help a shard that never advances.

B More shards

Redistributes future records and leaves the existing poison pill exactly where it is, still blocking its shard.

C Bound the retries and bisect the batch

maxRetryAttempts and maxRecordAge stop the infinite retry, and bisecting isolates the one bad record so the rest of the batch succeeds.

D Enhanced fan-out

Gives consumers dedicated throughput. It changes how records are delivered, not what happens when one always fails.

THE SIGNAL

'One shard stuck, others fine' is the poison pill signature. Ordering guarantees mean the shard cannot skip the failing record, so it retries forever until bounded.

> A newly deployed Lambda function fails on every invocation. The CloudWatch log group for the function exists but contains no log streams at all. The Errors metric is zero and the Invocations metric is zero, but the upstream caller reports failures.
>
> The function was working in the development account with an identical deployment.

Where should the team look first?

A Reserved concurrency set to zero, or an account concurrency limit, throttling every invocation

B A syntax error in the handler causing an initialisation failure

C The function timeout being too short for the workload

D A missing VPC endpoint preventing the function reaching DynamoDB

◆ PAUSE AND DECIDE

Where should the team look first?

A Concurrency at zero, or throttling

Invocations is zero and Errors is zero, so the function is not running at all. Reserved concurrency of zero throttles every call, which is exactly this signature.

B Syntax error

An initialisation failure still counts as an invocation and an error, and writes a log stream containing the stack trace. All three are absent.

C Timeout too short

A timeout increments Invocations and Errors and writes a log line. Nothing here is being recorded.

D Missing VPC endpoint

The function would run, fail to connect, and log a timeout. That produces invocations and errors, not silence.

THE SIGNAL

Invocations at zero is the decisive metric. Every other option requires the function to have run, and running always produces an invocation and a log stream.

What to carry forward

- **Retry 5xx, 429 and 408. Never retry other 4xx.** The permission will not appear.
- **Backoff without jitter still synchronises clients.** Jitter is what breaks the waves.
- **An inner timeout longer than an outer one never fires.** Layers must agree.
- Retries make duplicates inevitable, so idempotency is a requirement, not a refinement.
- **Invocations at zero means it never ran.** Look at throttling before looking at code.

> ON THE EXAM
>
> Next: performance and cost optimisation, which closes Domain 4 and the teaching content.